Links

https://huggingface.co/datasets/gtfintechlab/finer-ord
Financial NER dataset

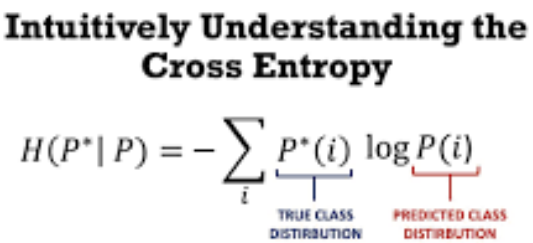
https://arxiv.org/abs/2302.11157
FiNER Paper

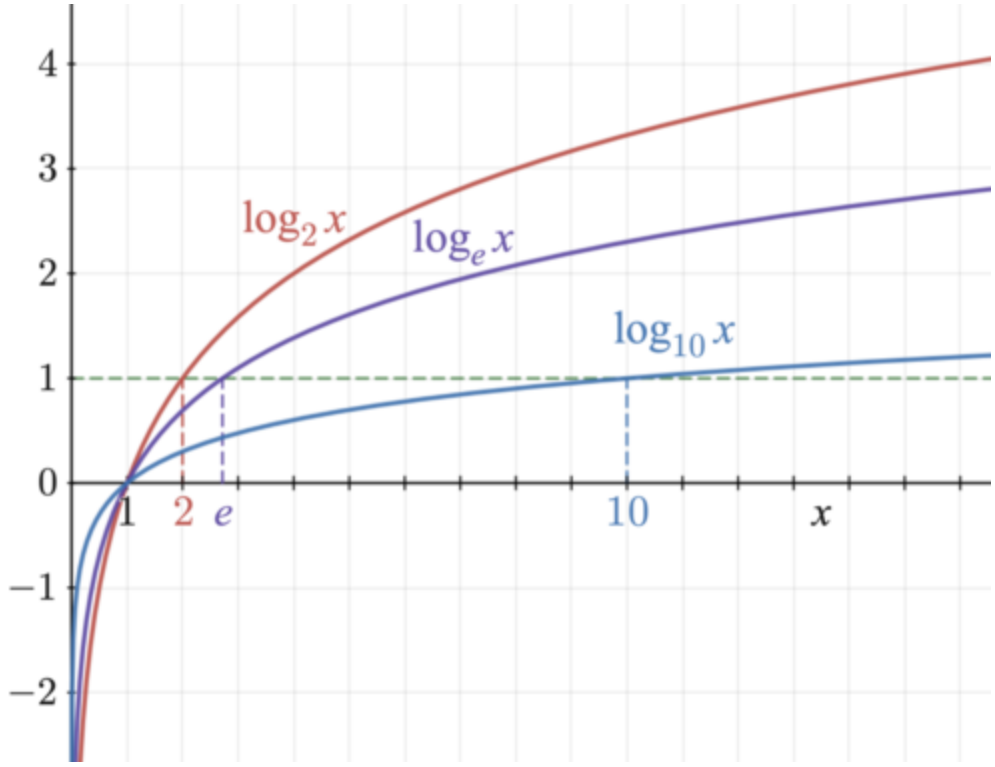
https://github.com/gtfintechlab/FiNER-ORD/blob/main/code/bert_for_ner_seqeval.py
https://github.com/gtfintechlab/FiNER-ORD/blob/main/code/news_fine_tune_ner_roberta_large.json
Code

Loss Function

- Cross entropy

$$L = -\sum_{k=1}^{K} y_k \log(p_k)$$





https://docs.pytorch.org/docs/2.11/generated/torch.nn.CrossEntropyLoss.html

- Predicted class distribution are the probabilities (taking softmax over the logits of the possible classes for that token in the sequence). CE does softmax for you hence p_k being prob confidence of the kth label.
- True class distribution is **NOT**! the weights you are trying to learn to minimize cross entropy. This is simply **a one hot encoded vector** with a 1 for the true label and else 0 for other labels so takes the negative log of the probability confidence for only the ground truth label (so can focus on penalizing for low probability on the ground truth label)
    - Can perform “label smoothing” where instead of a one hot encoded vector for the true class dist you have probs to weight the other probabilities too for the ground truth’s probability
    - Pytorch’s CE has a “**weight”** parameter which is just an additional vector, consisting of constants (they are not updated by optimizer), one for each label, and are multiplied by each label’s negative log prob. This is to help unbalanced classification datasets (like the FiNER dataset as 93% of the tokens are not an NER entity (the NER entity labels are so few compared to the non NER label) so all of the other labels would get a larger weight to penalize heavier for getting their samples wrong to avoid overfitting on the NER non entity label). This weight vector typically in relation to # of samples have for each label (unbalanced dataset)
    -
- Understanding negative log probability

- If a predicted prob confidence is lower then its negative log will be a larger positive number (penalized heavier for smaller confidence on ground truth label)
    - And because of the one hot encoding vector, this negative log prob is only accounted for the token's ground truth label (assuming not doing label smoothing)
- If a predicted prob confidence is high,the neg log will be a smaller positive number (penalized less for higher confidence on ground truth label)

$$\ell(x, y) = L = \{l_1, \dots, l_N\}^\top, \quad l_n = -w_{y_n} \log \frac{\exp(x_{n,y_n})}{\sum_{c=1}^{C} \exp(x_{n,c})} \cdot 1\{y_n \neq \text{ignore\_index}\}$$

-
    - The CE loss is the negative log prob of the ground truth label y_n and multiplied by optional weight w (which is a constant from the weight vector hyperparameter). If the ground truth label for the token is the ignore_index (-100 by defaut, then that loss is not considered)

$$\ell(x, y) = \begin{cases} \sum_{n=1}^{N} \frac{1}{\sum_{n=1}^{N} w_{y_n} \cdot 1\{y_n \neq \text{ignore\_index}\}} l_n, & \text{if reduction} = \text{'mean'}; \\ \sum_{n=1}^{N} l_n, & \text{if reduction} = \text{'sum'}. \end{cases}$$

-
    - Reduction = mean vs sum (loss across all of the samples in the batch)
- https://medium.com/@chris.p.hughes10/a-brief-overview-of-cross-entropy-loss-523aa56b75d5
- weight initialization (not for CE but for actual model's weights)
    - Uniform
    - xavier/glorot

cross entropy what is the true class distribution prob

In cross entropy, the "true" class distribution probability (P) represents the ground truth for your data. In most supervised learning classification tasks, this is represented using **one-hot encoding**.

- **The Target Class:** The probability is **1.0** for the correct class.
- **The Incorrect Classes:** The probability is **0.0** for all other classes.

$$\mathcal{L} = -\sum_{i=1}^{m}\sum_{j=1}^{n} y_{i,j}\log\left(p_{i,j}\right),$$

$$p = \frac{\exp\left(\text{net}_{i,j}\right)}{\exp\left(\text{net}_{i,1}\right) + \exp\left(\text{net}_{i,2}\right) + \cdots + \exp\left(\text{net}_{i,n}\right)}$$

Cross entropy (H) is calculated as:

$$H(P,Q) = -\sum_{i} P(i)\log(Q(i))$$

- $P(i)$ is the **true probability** (the 1 or 0 labels).
- $Q(i)$ is the **predicted probability** from your model (usually the output of a Softmax layer).

Because $P(i)$ is 0 for all classes except the correct one, the formula simplifies to just taking the negative log of the predicted probability for the actual correct class. This effectively "ignores" the predictions for the wrong classes and focuses entirely on how confident the model was in the right answer.

Alternative: Label Smoothing

In some advanced training scenarios, you might use **label smoothing**, where the true distribution is slightly modified (e.g., the true class is 0.9 and the others share the remaining 0.1). This helps prevent the model from becoming overconfident. [2]

so what does weights do in Pytorch's CE loss function

The `weight` parameter in PyTorch's `nn.CrossEntropyLoss` is a 1D tensor used to **manually rescale the loss for each class**. Its primary purpose is to help the model handle **unbalanced datasets**. PyTorch documentation +2

How it works mathematically

When you provide weights, the loss for each sample in a batch is multiplied by the specific weight assigned to its **true class**. PyTorch Forums +1

If your model is using the default `reduction='mean'`, PyTorch calculates the total loss as a **weighted average**. Instead of dividing by the total number of samples (N), it divides by the sum of the weights corresponding to each sample in that batch. PyTorch documentation +3

Key Benefits

- **Balancing importance:** You can assign higher weights to "minority" classes (those with fewer training images) so the network is punished more severely when it misclassifies them.
- **Controlling cost sensitivity:** It allows you to make certain errors more "expensive" than others based on real-world costs.
- **Alternative to oversampling:** It adjusts the learning process directly without needing to duplicate data or modify your dataset structure. PyTorch Forums +3

Implementation Example

A typical choice for these values is the **inverse of the class frequency**: Stack Overflow +1

python

```python
# If Class A has 100 samples and Class B has 1000
weights = torch.tensor([1.0, 0.1])
criterion = nn.CrossEntropyLoss(weight=weights)
```

so after taking the log prob of the true label, you multiply it by the weight?

Exactly. It's a simple scaling step at the very end of the calculation for each sample.

If you have a sample belonging to **Class i**, the loss for that specific sample becomes:

$$\text{Loss} = -\text{weight}[i] \cdot \log(Q(i))$$

What this looks like in practice:

- **High Weight (e.g., 5.0):** If the model misses a rare class, the loss is 5x larger than normal. This forces the optimizer to make a much bigger update to the weights to "fix" that mistake.
- **Low Weight (e.g., 0.1):** If the model misses a very common class, the loss is tiny. The model won't stress too much about these errors, preventing the common class from dominating the learning process.

The "Weighted Mean"

It's important to note that when PyTorch calculates the **average** loss for the whole batch, it doesn't just divide by the number of samples (N). It divides by the **sum of the weights** of the samples in that batch. This keeps the loss on a similar scale regardless of how large your weights are.

are these weights updated or are constant

In PyTorch's `nn.CrossEntropyLoss`, the weights are **constant**.

They are hyperparameters that you define before training begins. Unlike the weights inside your neural network layers, the loss function weights are not updated by backpropagation or the optimizer.

1. Manual definition

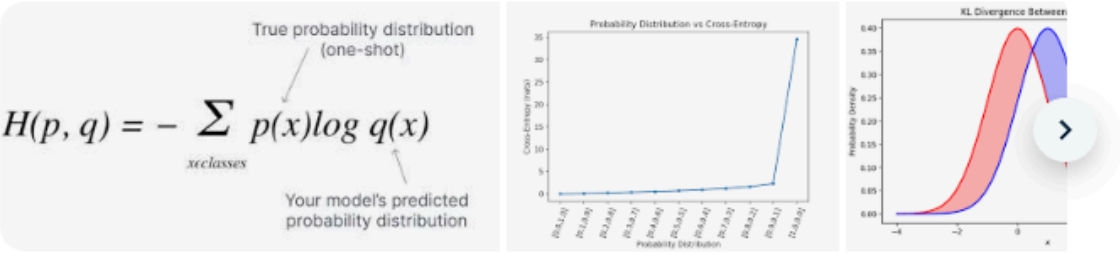
You typically calculate these weights once based on your training data distribution and pass them to the loss function. They remain fixed for the entire duration of the training process.

2. Purpose of being constant

The weights act as a "rule of the game." If they changed during training, the "importance" of each class would be a moving target, which would make the optimization process extremely unstable and difficult to converge.

3. When they *might* change

While the weights are constant by default, some advanced researchers use **Custom Training Loops** to manually adjust them between epochs (often called "Cost-Sensitive Learning" or "Schedules"). However, this is not built into the standard PyTorch loss function; you would have to manually update the `criterion.weight` attribute yourself.



In machine learning, the true class distribution (often denoted as P or y) in cross-entropy is the actual, ground-truth label of a data point, represented as a one-hot encoded vector. It assigns 100% probability to the correct class and 0% to all others, forcing the predicted probability distribution to match this target for minimal loss. Pinecone +3

Key Details on True Class Distribution (P):

- **One-Hot Encoding:** If a dataset has three classes (cat, dog, bird) and the target is "dog," the true distribution is $[0, 1, 0]$.
- **Representation:** For a multiclass problem, the true distribution is a vector of length N (number of classes), containing all the probability mass (1) at the index of the correct class.
- **Formula Context:** In the cross-entropy formula $H(P, Q) = -\sum P(i)\log(Q(i))$, $P(i)$ represents the true label (1 or 0), while $Q(i)$ is the predicted probability.
- **Purpose:** The goal is to make the predicted probability Q of the true class as close to 1 as possible, minimizing the loss. Pinecone +4

If the model predicts $[0.7, 0.2, 0.1]$ for the classes [dog, cat, bird] but the true class is "cat", the true distribution is $[0, 1, 0]$, causing the cross-entropy to penalize the model for its low (0.2) confidence in the correct label. Weights & Biases +2

-

Yes. In your case, the class weights enter CrossEntropyLoss by multiplying the loss from each token by the weight of its true class.

For one token with true class y and predicted logits $z \in \mathbb{R}^C$, the weighted cross entropy is:

$$\ell = -w_y \log\left(\frac{e^{z_y}}{\sum_{c=1}^{C} e^{z_c}}\right)$$

where:

- C is the number of classes
- w_y is the weight for the true class y

So if a class has a larger weight, mistakes on that class count more. If a class has a smaller weight, mistakes on that class count less.

For your token-classification setup, the model produces logits for every token in the sequence, and the total loss is basically the average over all non-ignored tokens:

$$L = \frac{\sum_{n=1}^{N} \ell_n}{N}$$

-

$$L = \frac{\sum_{n=1}^{N} \ell_n}{N}$$

with tokens labeled -100 ignored entirely.

In your code, the weight vector is:

$$w = [w_0, w_1, w_2, w_3, w_4, w_5, w_6]$$

where each position matches one class index:

- w_0 for O
- w_1 for PER_B
- w_2 for PER_I
- w_3 for LOC_B
- w_4 for LOC_I
- w_5 for ORG_B
- w_6 for ORG_I

So when the gold label for a token is, say, ORG_B, PyTorch uses w_5 to scale that token's contribution to the loss.

-

does crss entopy reduciton mean find mean across all tokens (batchsize*seq_len)

AI Overview

Yes, when using Cross-Entropy Loss with `reduction='mean'` (the default in PyTorch), it means finding the average loss across all non-ignored tokens—effectively dividing the total sum of losses by the total number of tokens in the batch ($batch\_size \times sequence\_length$). PyTorch Forums +2

Key Details on Reduction Methods

- `reduction='mean'` **(Default):** The sum of the loss is divided by the number of elements in the output. In NLP tasks with `(batch_size, sequence_length)`, this is the mean over all tokens.
- `reduction='sum'`: The losses are summed, not averaged.
- `reduction='none'`: Returns the loss for each individual token (a tensor of shape `(batch_size, sequence_length)`). PyTorch +4

Why "Mean" is Usually Used

1. **Independent of Batch Size:** The average loss ensures that if you change the batch size, the magnitude of the gradients remains roughly the same, eliminating the need to adjust the learning rate.
2. **Equivalent to Token-Level Averaging:** It ensures every token, regardless of the sentence it belongs to, contributes equally to the training objective. PyTorch Forums +2

Special Considerations (Masking)

If your input includes padding tokens and you use `ignore_index` in the loss function, the `reduction='mean'` will **only average over the non-masked tokens**. It divides by the count of non-ignored targets rather than the total tensor shape. PyTorch Forums +1

-
    - reduction = mean
    - Note we pass in (batchsize * seq_len, num_labels) for the logits and (batch_size * seq_len) for the labels -> (condense the batch and sequence dimensions)
        - so the average is over total # of tokens in the entire batch
- Cross entropy (CE) is loss for multiclass classification
    - CE also for Autoregressive language model training (next location's token prediction can be seen as multi-class classification where the logits across the vocab_size dimension are the confidence scores for predicting the token in the next location
    - Used to measure confidence in expected correct class

For multi-class classification, the standard and most effective loss function is **Categorical Cross-Entropy (CCE)**. It measures the difference between the true label distribution (often one-hot encoded) and the predicted probability distribution generated by your model. GeeksforGeeks +3

-
- CE and NLL (neg log likelihood) both used for multi class classification / language training and are vice versa (can use either and result in equivalent result. Just need to handle last layer differently)
  - CE expects raw logits (no softmax)
  - NLL expects the last layer in model to be logsoftmax (and make sure to perform on last dimension to get probabilites for every individual token)
-

```
127             self.log_softmax = nn.LogSoftmax(dim=-1) # NLLLoss expects log-probabilities as in
```

```
100     class TransformerNextToken(torch.nn.Module):
149         def forward(self, indices: torch.Tensor):
                x = self.char_embedding(indices)
                x = self.positional_encoding(x) # adds positional encoding to char embeddings (see transformer.py)

                """
                make mask dynamic so can avoid having to pad by using the input sequence length rather than model's seq_ler
                - embedding layer simply converts given indices to their embedding vector representation
                - only layer seq_len matters is for PositionalEncoding
                - so for positional encoding's embedding layer, there isnt a problem if input sequence length is shorter th
                   - seq_len is paramater of positional embedding layer but just specifies its lookup table size
                - but input sequence length is a problem if larger than seq_len (outside of positional embedding's vocab)
                where you would need to truncate input sequence to last seq_len tokens
                """
                input_seq_len = x.shape[1]
                mask=torch.nn.Transformer.generate_square_subsequent_mask(sz=input_seq_len).to(x.device) # (input_seq_len,
                x = self.transformer_encoder(x, mask=mask, is_causal=True) # (batch_size, seq_len, d_model)
                x = self.linear_out(x) # (batch_size, seq_len, vocab_size)
                x = self.log_softmax(x) # (batch_size, seq_len, vocab_size), log-probabilities for NLLLoss
                return x
```

-
  - `nn.CrossEntropyLoss` **(CE)** expects **raw logits** (unnormalized, unbounded scores) as input. It automatically applies a `log_softmax` operation followed by the negative log-likelihood inside the loss function for numerical stability.
  - `nn.NLLLoss` **(NLL)** requires **log-probabilities** (log-softmax) as input. It does not calculate the log part itself; it only performs the negative likelihood part. GitHub +5

  Key Takeaways:

  - **Do not** apply `softmax` before `nn.CrossEntropyLoss`.
  - **Do** apply `log_softmax` to the last layer if you are using `nn.NLLLoss`.
  - `nn.CrossEntropyLoss` = `nn.LogSoftmax` + `nn.NLLLoss`. Medium +3

  Why?
  Using raw logits with `CrossEntropyLoss` is preferred because it handles the `log` and `softmax` computations together in a single, more numerically stable step (using the log-sum-exp trick), rather than calculating softmax and logarithm separately. GitHub +1
-

- So we use CE (or NLL) also for NER task, on fine tuning of a pretraining LLM bc NER is a multi class classification task

```
random.shuffle(train_chunks)
# NOTE: does not matter if chunk first then batch second or vice versa.
# Actually easier if create chunks first then batch second similar to transformer.py and its given dataset
for b in range(0, len(train_chunks), batch_size):
    batch_chunks = train_chunks[b:b+batch_size]
    # make sure save different tensors for model input and target since preprocess_input here will shift t
    # dont want targets to be shifted too or model will be cheating -> will lead to perplexity of around 1
    # can see test in lm.py: error if perplexity < 3.5
    batch_tensor_targets = torch.tensor(np.array(batch_chunks), dtype=torch.long).to(device) # (batch_size
    batch_tensor_train = model.preprocess_input(batch_tensor_targets, training=True) # (batch_size, seq_le
    logits = model(batch_tensor_train) # (batch_size, seq_len, vocab_size)
    # Collapse batch dim for NLL loss funct (would also have to do this if using cross entropy)
    logits = logits.reshape(-1, logits.shape[-1]) # (batch_size * seq_len, vocab_size)
    targets = batch_tensor_targets.reshape(-1) # (batch_size * seq_len,)
    # Ground truth targets are the token index (from vocab indexer) expected at each position
    loss = loss_fn(logits, targets)
    loss.backward()
    optimizer.step()
    model.zero_grad()
```

- Here (my transformers_LM assignment) using NLL loss but same idea of reshaping output from transformer and labels before passing into loss funct

```
258                 for input_ids, attention_masks, labels in tqdm(dataloaders_dict[phase]):
259                     input_ids = input_ids.to(self.device)
260                     attention_masks = attention_masks.to(self.device)
261                     labels = labels.to(self.device)
262                     optimizer.zero_grad()
263                     with torch.set_grad_enabled(phase == 'train'):
264                         outputs = model(input_ids=input_ids, attention_mask=attention_masks, labels=labels)
265                         active_loss = attention_masks.view(-1) == 1
266                         logits = outputs.logits
267                         active_logits = logits.view(-1, num_labels)
268                         active_labels = torch.where(
269                             active_loss, labels.view(-1), torch.tensor(criterion.ignore_index).type_as(labels)
270                         )
271                         loss = criterion(active_logits, active_labels)
272
```

- From FiNER paper's code
- .view() to make the shape basically (batch*seq_len, num_NER_labels)
- Note we pass a labels hyperparam to model()
    - Question is how for a language model we're changing the last dimension shape: from
        - (batch_size, seq_len, vocab_size)
        - to
        - (batch_size, seq_len, num_NER_labels)
    - Could do it simply with an additional MLP block (linear layer can transform the last dim)

```
if self.language_model == 'bert-base-cased':
    model = BertForTokenClassification.from_pretrained('bert-base-cas
elif self.language_model == 'roberta-base':
    model = RobertaForTokenClassification.from_pretrained('roberta-ba
elif self.language_model == 'SALT-NLP/FLANG-Roberta':
    model = RobertaForTokenClassification.from_pretrained('SALT-NLP/F
elif self.language_model == 'finbert-cased':
    model = BertForTokenClassification.from_pretrained('../finbert-ca
elif self.language_model == 'SALT-NLP/FLANG-BERT':
    model = BertForTokenClassification.from_pretrained('SALT-NLP/FLAN
elif self.language_model == 'bert-large-cased':
    model = BertForTokenClassification.from_pretrained('bert-large-ca
elif self.language_model == 'roberta-large':
    model = RobertaForTokenClassification.from_pretrained('roberta-la
elif self.language_model == 'xlnet-base-cased':
    model = XLNetForTokenClassification.from_pretrained("xlnet-base-c
optimizer = optim.AdamW(model.parameters(), lr=lr)
```

```
17  from transformers import RobertaForTokenClassification, RobertaTokenizerFast, BertForTokenClassificatic
```

- model variable is from hugging face library
- https://huggingface.co/docs/transformers/v5.6.2/en/model_doc/roberta#transformers.RobertaForTokenClassification

  > The Roberta transformer with a token classification head on top (a linear layer on top of the hidden-states output) e.g. for Named-Entity-Recognition (NER) tasks.

-
- So I was right, for this NER task, basically take a pretrained transformer and then now to perform this new classification task, transform the logits last dim using a linear layer from vocab_size to num_NER_labels

Hugging Face Library Transformers Understanding & Glossary Terms

https://huggingface.co/docs/transformers/v5.6.2/en/model_doc/roberta#transformers.RobertaForTokenClassification.forward

Note using “Token” Classification class which is what NER task is

NER task is NOT a sequence to sequence task

- Input_ids
  - https://huggingface.co/docs/transformers/v5.6.2/en/model_doc/roberta#transformers.RobertaForTokenClassification.forward.input_ids

    input_ids (`torch.LongTensor` of shape `(batch_size, sequence_length)`, *optional*) — Indices of input sequence tokens in the vocabulary. Padding will be ignored by default.
  - https://huggingface.co/docs/transformers/v5.6.2/en/glossary#input-ids
  - Indices of input sequence tokens in the vocabulary. Padding will be ignored by default. NOTE the tokenizer will automatically add special tokens too
    - The “vocab” is the LM’s (language model’s) vocab (remember from its embedding layer is/has a vocabulary)
    - Each hugging face pretrained LM is identified by a string (like “roberta-large”) and has a tokenizer and model class
      - Tokenizer
        - tokenizer.tokenize() - separates string of text into tokens (subwords or words) based off how that LM does its tokenizing
        - tokenizer(tokens) - returns object with needed attributes like input_ids and attention_mask, etc.

```
>>> tokenizer = BertTokenizer.from_pretrained("google-bert/bert-base-cased")

>>> sequence = "A Titan RTX has 24GB of VRAM"
```

The tokenizer takes care of splitting the sequence into tokens available in the tokenizer vocabulary.

```
>>> tokenized_sequence = tokenizer.tokenize(sequence)
```

The tokens are either words or subwords. Here for instance, “VRAM” wasn’t in the model vocabulary, so it’s been split in “V”, “RA” and “M”. To indicate those tokens are not separate words but parts of the same word, a double-hash prefix is added for “RA” and “M”:

```
>>> print(tokenized_sequence)
['A', 'Titan', 'R', '##T', '##X', 'has', '24', '##GB', 'of', 'V', '##RA', '##M']
```

        -
          - Note above that the tokenizer still considers tokens that are not in the LM’s vocab

These tokens can then be converted into IDs which are understandable by the model. This can be done by directly feeding the sentence to the tokenizer, which leverages the Rust implementation of 🤗 Tokenizers for peak performance.

```
>>> inputs = tokenizer(sequence)
```

The tokenizer returns a dictionary with all the arguments necessary for its corresponding model to work properly. The token indices are under the key `input_ids`:

```
>>> encoded_sequence = inputs["input_ids"]
>>> print(encoded_sequence)
[101, 138, 18696, 155, 1942, 3190, 1144, 1572, 13745, 1104, 159, 9664, 2107, 102]
```

Note that the tokenizer automatically adds "special tokens" (if the associated model relies on them) which are special IDs the model sometimes uses.

If we decode the previous sequence of ids,

```
>>> decoded_sequence = tokenizer.decode(encoded_sequence)
```

we will see

```
>>> print(decoded_sequence)
[CLS] A Titan RTX has 24GB of VRAM [SEP]
```

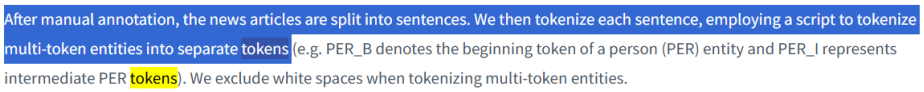
because this is the way a BertModel is going to expect its inputs.

- Model
  - Pass the input_ids, attention_mask, etc and get output logits

- If you have a string of text, you would first tokenize it by passing the text into the tokenizer.tokenize() method (turning string of text into list of individual tokens).
- Then with the tokenized input, you would acquire their input_ids (indices associated with the model's vocab) by passing the list of tokens into the tokenizer's forward method.
- This organizes the input text into tokens and input_ids so will get (batch_size, seq_len) where seq_len are the tokens in that sequence (in our case a sentence in the FiNER dataset)
- Each LM can have their own way of tokenizing which is why we call the tokenizer's tokenize method and dont pass raw text into the model's forward
- Ex:
- "I am walking to the store"
- Could end up becoming
- ["I", "am", "walking", "to", "the", "store"] and then it's input_ids would be say
- [14, 144, …etc]

```
99        def load_data(self, file_path: str, int2str: Dict[int, str]):
107
108           Description: Data must contain these three columns.
109               (1) sentence_id: Index of the sentence
110               (2) token: token of sentence
111               (3) label: NER label for token
112
113           """
114           df = pd.read_csv(file_path)
115           # print(file_path)
116           # print(df.head())
117           df.label = df.label.map(int2str)
118           df.dropna(inplace=True)
119
120           # df_sentences = df.groupby('uuid').agg({'token': list, 'label': list})
121           df_sentences = df.groupby(['document_id', 'sentence_id']).agg({'token': list, 'label': list})
122           # print(df_sentences.head())
123           sentences = df_sentences.token.tolist()
124           sentences_tags = df_sentences.label.tolist()
```

```
159           tokenized_inputs = self.tokenizer(sentences,
160                                             max_length=max_length,
161                                             padding='max_length',
162                                             is_split_into_words=True,
163                                             return_tensors='pt')
164           input_ids = tokenized_inputs['input_ids']
165           attention_masks = tokenized_inputs['attention_mask']
```

- Note the FiNER dataset is already split into tokens ,
  - and each sequence is a sentence
  - so question is if the tokens being split without using the tokenizer's tokenize() method is a problem
    - Because we know the NER task is basically words as tokens, then it's important to flag is_split_into_words as True for the tokenizer()
      - Also note with this flag a word can still be tokenized into multiple sub words (so do not expect that the tokenization will result in the original full words, some words will still be tokenized into subwords)
    - 

    - note that multi token entities are split into separate tokens so the dataset has been tokenized by words for sure

```
52            if self.language_model == 'bert-base-cased':
53                self.tokenizer = BertTokenizerFast.from_pretrained('bert-base-cased', do_lower_case=False
54            elif self.language_model == 'roberta-base':
55                self.tokenizer = RobertaTokenizerFast.from_pretrained('roberta-base', do_lower_case=False
56            elif self.language_model == 'SALT-NLP/FLANG-Roberta':
57                self.tokenizer = RobertaTokenizerFast.from_pretrained('SALT-NLP/FLANG-Roberta', do_lower_
58            elif self.language_model == 'finbert-cased':# https://github.com/yya518/FinBERT
59                self.tokenizer = BertTokenizerFast(vocab_file='../finbert-cased/FinVocab-Cased.txt', do_l
60            elif self.language_model == 'SALT-NLP/FLANG-BERT':
61                self.tokenizer = BertTokenizerFast.from_pretrained('SALT-NLP/FLANG-BERT', do_lower_case=F
62            elif self.language_model == 'bert-large-cased':
63                self.tokenizer = BertTokenizerFast.from_pretrained('bert-large-cased', do_lower_case=Fals
64            elif self.language_model == 'roberta-large':
65                self.tokenizer = RobertaTokenizerFast.from_pretrained('roberta-large', do_lower_case=Fals
66            elif self.language_model == 'xlnet-base-cased':
67                self.tokenizer = XLNetTokenizerFast.from_pretrained("xlnet-base-cased", do_lower_case=Fal
68
```

```
503        def grid_search_bert(self):
531                    if self.language_model == 'bert-base-cased':
532                        model = BertForTokenClassification.from_pretrained('bert-base-cased', num_labels=7).to(self.device)
533                    elif self.language_model == 'roberta-base':
534                        model = RobertaForTokenClassification.from_pretrained('roberta-base', num_labels=7).to(self.device)
535                    elif self.language_model == 'SALT-NLP/FLANG-Roberta':
536                        model = RobertaForTokenClassification.from_pretrained('SALT-NLP/FLANG-Roberta', num_labels=7).to(self.devi
537                    elif self.language_model == 'finbert-cased':
538                        model = BertForTokenClassification.from_pretrained('../finbert-cased/model', num_labels=7).to(self.device)
539                    elif self.language_model == 'SALT-NLP/FLANG-BERT':
540                        model = BertForTokenClassification.from_pretrained('SALT-NLP/FLANG-BERT', num_labels=7).to(self.device)
541                    elif self.language_model == 'bert-large-cased':
542                        model = BertForTokenClassification.from_pretrained('bert-large-cased', num_labels=7).to(self.device)
543                    elif self.language_model == 'roberta-large':
544                        model = RobertaForTokenClassification.from_pretrained('roberta-large', num_labels=7).to(self.device)
545                    elif self.language_model == 'xlnet-base-cased':
546                        model = XLNetForTokenClassification.from_pretrained("xlnet-base-cased", num_labels=7).to(self.device)
547                    optimizer = optim.AdamW(model.parameters(), lr=lr)
548
549                    self.fine_tune(model, optimizer, dataloaders_dict, train_str2int)
550                    self.test(model, optimizer, dataloaders_dict, train_str2int)
551                    # self.test(model, optimizer, dataloaders_dict, test_str2int)
552
553
554    def main():
555        config_path = sys.argv[1]
556
557        bert_for_ner_setup = BERTForNer(config_path)
558        bert_for_ner_setup.grid_search_bert()
```

- Note the num_labels param, probably indicates the hidden dim size for the last linear layer in the model (turn vocab_size dim into num_labels) for our NER task
- IMPORTANT to know that each hugging face LM has a model and a tokenizer. They use the same string name for both but are different classes to create instances of and use
- Tokenizer is to format text input into tokens and acquire the indices for the tokens in terms of the LM's vocab lookup table (proper format before passing into model)
- Model is the actual transformer/model

- https://github.com/gtfintechlab/FiNER-ORD/blob/main/code/bert_for_ner_seqeval.py#L159

```
164             input_ids = tokenized_inputs['input_ids']
165             attention_masks = tokenized_inputs['attention_mask']
166             labels = []
167             temp_global_list_labels = set()
168             for i, label in enumerate(sentences_tags):
169                 word_ids = tokenized_inputs.word_ids(batch_index=i)
170                 previous_word_idx = None
171                 label_ids = []
172                 for word_idx in word_ids:
173
174                     if word_idx is None:
175                         label_ids.append(-100)
176                     elif word_idx != previous_word_idx:
177                         label_ids.append(str_to_int[label[word_idx]])
178                     else:
179                         label_ids.append(str_to_int[label[word_idx]] if self.label_all_tokens else -100)
180                     previous_word_idx = word_idx
181                 labels.append(label_ids)
182             labels = torch.LongTensor(labels)
183             # return TensorDataset(input_ids, attention_masks, labels, doc_indices_tensor, sent_indices_tensor), st
184             return TensorDataset(input_ids, attention_masks, labels), str_to_int, int_to_str
```

- here but after tokenizing our input, we have to pair each token with its ground truth NER label to prepare labels in conjunction with the tokenizer's input_ids (how the tokenizer tokenized)
  - Remember, tokens in this dataset were already split into tokens manually (by words) which could possibly not be in the model's vocab
  - So I think here if a word is not in the vocab or is a padding token, then it's word_idx is None (maybe) so for our labels list, we just flag the token with -100 to indicate to not consider it when computing loss and back prop
    - CE function's ignore_index param is -100 by default
    - So -100 indicates to ignore that token when backprop/compute loss

- Attention_mask

  attention_mask (`torch.FloatTensor` of shape `(batch_size, sequence_length)`, *optional*) — Mask to avoid performing attention on padding token indices. Mask values selected in `[0, 1]`:
  1 for tokens that are **not masked**,
  0 for tokens that are **masked**.
  - What are attention masks?
  - https://huggingface.co/docs/transformers/v5.6.2/en/glossary#attention-mask
  - For batching purposes and dealing with sequences of different lengths in the same tensor (batch)

- If dealing with sequences of not same length, must flag padding hyperparam when tokenizing so the tokenizer can pad so all sequences are same length, and then the tokenizer will spit out an attention mask with 0s where it padded in the sequences and 1 elsewhere
- Attention mask helps tell model which tokens were padding (not to consider)
- Tokenizer
- https://huggingface.co/docs/transformers/en/main_classes/tokenizer#transformers.PythonBackend.__call__
    -
```
159            tokenized_inputs = self.tokenizer(sentences,
160                                              max_length=max_length,
161                                              padding='max_length',
162                                              is_split_into_words=True,
163                                              return_tensors='pt')
164            input_ids = tokenized_inputs['input_ids']
165            attention_masks = tokenized_inputs['attention_mask']
```
- **padding** (`bool`, `str` or PaddingStrategy, *optional*, defaults to `False`) — Activates and controls padding. Accepts the following values:
`True` or `'longest'`: Pad to the longest sequence in the batch (or no padding if only a single sequence is provided).
`'max_length'`: Pad to a maximum length specified with the argument `max_length` or to the maximum acceptable input length for the model if that argument is not provided.
`False` or `'do_not_pad'` (default): No padding (i.e., can output a batch with sequences of different lengths).
    - Note padding=true vs. padding='max_length'
- **max_length** (`int`, *optional*) — Controls the maximum length to use by one of the truncation/padding parameters.
If left unset or set to `None`, this will use the predefined model maximum length if a maximum length is required by one of the truncation/padding parameters. If the model has no specific maximum input length (like XLNet) truncation/padding to a maximum length will be deactivated.

is_split_into_words (`bool`, *optional*, defaults to `False`) — Whether or not the input is already pre-tokenized (e.g., split into words). If set to `True`, the tokenizer assumes the input is already split into words (for instance, by splitting it on whitespace) which it will tokenize. This is useful for NER or token classification.

- Important note the FiNER dataset was already tokenized (by words) so we flag this as True

return_tensors (`str` or TensorType, *optional*) — If set, will return tensors instead of list of python integers. Acceptable values are:
`'pt'`: Return PyTorch `torch.Tensor` objects.
`'np'`: Return Numpy `np.ndarray` objects.

Hugging Face tokenizers set the attention mask to **0 ONLY for padding tokens**, not for special tokens like `[CLS]`, `[SEP]`, or `<bos>`. Special tokens are considered part of the active sequence and receive a **1** in the mask, allowing the model to attend to them, while padding is ignored. Hugging Face +2

- **1 (Attend):** Real tokens, Special tokens (e.g., `<s>`, `<pad>` is not included), Beginning of sentence.
- **0 (Ignore):** Padding tokens. Hugging Face +3

The attention mask ensures that the model ignores padding tokens, as they are only added to ensure all sequences in a batch have the same length. Hugging Face

- Labels
  - https://huggingface.co/docs/transformers/v5.6.2/en/glossary#labels
    - For token classification models, (BertForTokenClassification), the model expects a tensor of dimension `(batch_size, seq_length)` with each value corresponding to the expected label of each individual token.

The RobertaForTokenClassification forward method, overrides the `__call__` special method.

> Although the recipe for forward pass needs to be defined within this function, one should call the `Module` instance afterwards instead of this since the former takes care of running the pre and post processing steps while the latter silently ignores them.

- **loss** (`torch.FloatTensor` of shape `(1,)`, *optional*, returned when `labels` is provided) — Classification loss.

- **logits** (`torch.FloatTensor` of shape `(batch_size, sequence_length, config.num_labels)`) — Classification scores (before SoftMax).

```
                    with torch.set_grad_enabled(phase == 'train'):
                        outputs = model(input_ids=input_ids, attention_mask=attention_masks, labels=labels)
                        active_loss = attention_masks.view(-1) == 1
                        logits = outputs.logits
                        active_logits = logits.view(-1, num_labels)
                        active_labels = torch.where(
                            active_loss, labels.view(-1), torch.tensor(criterion.ignore_index).type_as(labels)
                        )
                        loss = criterion(active_logits, active_labels)

                        if phase == 'train':
                            loss.backward()
                            optimizer.step()
                        else:
                            curr_pred = outputs.logits.argmax(dim=-1).detach().cpu().clone().numpy()
                            curr_actual = labels.detach().cpu().clone().numpy()
```

- Not sure why labels is passed into the model when we calculate the loss ourselves
- Criterion is cross entropy loss function
- But important to note that since dealing with sequences of different lengths, we used padding, so must make sure we ignore the tokens that are padding tokens when computing loss
  - Active loss is boolean of True (use that token) and False if the attention mask was 0 (indicating it was a padding token)
    - Also note .view(-1) condenses the last dimension (batch_size, seq_len) to (batch_size * seq_len) which we need to do for the CE loss
  - Then active_labels is computed using active_loss
    - **Syntax:** `torch.where(condition, x, y)`
    - **Behavior:** $out_i = x_i$ if $condition_i$ is **True**, else $out_i = y_i$.

    - So if the token wasnt a padding token its the actual label, and if was padding token then is cross entropy's ignore index which is -100:
    - https://docs.pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html#torch.nn.CrossEntropyLoss
    - > **ignore_index** (*int*, *optional*) – Specifies a target value that is ignored and does not contribute to the input gradient. When `size_average` is `True`, the loss is averaged over non-ignored targets. Note that `ignore_index` is only applicable when the target contains class indices.
    - Ignore_index by default is -100
    - So if attention_mask had value of 0 for a token, then its label is going to be -100 indicating to ignore that token when backprop (updating gradients)
- FineTuning (ignore)
    - https://huggingface.co/docs/transformers/v5.6.2/en/glossary#finetuned-models
    - https://huggingface.co/docs/transformers/training#fine-tuning
    - Uses Trainer but paper's code does not (dont have to use Trainer)
- Head (ignore)
    - last block to transform last dimension of language model for classification tasks
    - https://huggingface.co/docs/transformers/v5.6.2/en/glossary#head
- Position_ids (ignore)
    - **position IDs**

      Contrary to RNNs that have the position of each token embedded within them, transformers are unaware of the position of each token. Therefore, the position IDs (`position_ids`) are used by the model to identify each token's position in the list of tokens.

      They are an optional parameter. If no `position_ids` are passed to the model, the IDs are automatically created as absolute positional embeddings.

      Absolute positional embeddings are selected in the range `[0, config.max_position_embeddings - 1]`. Some models use other types of positional embeddings, such as sinusoidal position embeddings or relative position embeddings.
    - https://huggingface.co/docs/transformers/v5.6.2/en/glossary#position-ids
- Token_type_ids
    - https://huggingface.co/docs/transformers/v5.6.2/en/glossary#token-type-ids

- Attributes from forward

  - **loss** (`torch.FloatTensor` of shape `(1,)`, *optional*, returned when `labels` is provided) — Classification loss.

  - **logits** (`torch.FloatTensor` of shape `(batch_size, sequence_length, config.num_labels)`) — Classification scores (before SoftMax).

  - **hidden_states** (`tuple(torch.FloatTensor)`, *optional*, returned when `output_hidden_states=True` is passed or when `config.output_hidden_states=True`) — Tuple of `torch.FloatTensor` (one for the output of the embeddings, if the model has an embedding layer, + one for the output of each layer) of shape `(batch_size, sequence_length, hidden_size)`.

    Hidden-states of the model at the output of each layer plus the optional initial embedding outputs.

  - **attentions** (`tuple(torch.FloatTensor)`, *optional*, returned when `output_attentions=True` is passed or when `config.output_attentions=True`) — Tuple of `torch.FloatTensor` (one for each layer) of shape `(batch_size, num_heads, sequence_length, sequence_length)`.

    Attentions weights after the attention softmax, used to compute the weighted average in the self-attention heads.

- ' Example:

```
>>> from transformers import AutoTokenizer, RobertaForTokenClassification
>>> import torch

>>> tokenizer = AutoTokenizer.from_pretrained("FacebookAI/roberta-base")
>>> model = RobertaForTokenClassification.from_pretrained("FacebookAI/roberta-base")

>>> inputs = tokenizer(
...     "HuggingFace is a company based in Paris and New York", add_special_tokens=False, r
... )

>>> with torch.no_grad():
...     logits = model(**inputs).logits

>>> predicted_token_class_ids = logits.argmax(-1)

>>> # Note that tokens are classified rather then input words which means that
>>> # there might be more predicted token classes than words.
>>> # Multiple token classes might account for the same word
>>> predicted_tokens_classes = [model.config.id2label[t.item()] for t in predicted_token_cl
>>> predicted_tokens_classes
...

>>> labels = predicted_token_class_ids
>>> loss = model(**inputs, labels=labels).loss
>>> round(loss.item(), 2)
...
```

- NOTE the logits attribute is defined as the “classification scores”

- This is bc BERT models are used for classification tasks:

  Yes, **BERT** (Bidirectional Encoder Representations from Transformers) is extensively used for classification tasks after being pre-trained as a language model. While it is a language model based on the transformer architecture, it is technically an encoder-only model designed specifically for understanding context, rather than generating text. Wikipedia +3

  ## How BERT is Used for Classification

  BERT is rarely used "raw" for classification. Instead, it follows a two-stage process:

  - **Pre-training:** The model learns language representations from large, unlabeled text data.
  - **Fine-tuning:** A task-specific classification layer is added on top of the pre-trained BERT model, which is then updated on a small, labeled dataset. Wikipedia +4

  ## Key Classification Use Cases

  BERT excels at tasks requiring deep semantic understanding, including:

  - **Sentiment Analysis:** Determining if a text is positive, negative, or neutral.
  - **Topic Classification:** Categorizing text into topics.
  - **Named Entity Recognition (NER):** Identifying tokens like names, places, or products.
  - **Question Answering:** Extracting answers from text. Coursera +4

- **Note BERT is used for NER classification task**

deep learning (DL)

Machine learning algorithms which use neural networks with several layers.

E

encoder models

Also known as autoencoding models, encoder models take an input (such as text or images) and transform them into a condensed numerical representation called an embedding. Oftentimes, encoder models are pretrained using techniques like masked language modeling, which masks parts of the input sequence and forces the model to create more meaningful representations.

feature extraction

The process of selecting and transforming raw data into a set of features that are more informative and useful f machine learning algorithms. Some examples of feature extraction include transforming raw text into word embeddings and extracting important features such as edges or shapes from image/video data.

feed forward chunking

In each residual attention block in transformers the self-attention layer is usually followed by 2 feed forward layers. The intermediate embedding size of the feed forward layers is often bigger than the hidden size of the model (e.g., for `google-bert/bert-base-uncased`).

For an input of size `[batch_size, sequence_length]`, the memory required to store the intermediate feed forward embeddings `[batch_size, sequence_length, config.intermediate_size]` can account for a large fraction of the memory use. The authors of Reformer: The Efficient Transformer noticed that since the computation is independent of the `sequence_length` dimension, it is mathematically equivalent to compute the output embeddings of both feed forward layers `[batch_size, config.hidden_size]_0, ..., [batch_size, config.hidden_size]_n` individually and concat them afterward to `[batch_size, sequence_length, config.hidden_size]` with `n = sequence_length`, which trades increased computation time against reduced memory use, but yields a mathematically **equivalent** result.

For models employing the function apply_chunking_to_forward(), the `chunk_size` defines the number of output embeddings that are computed in parallel and thus defines the trade-off between memory and time complexity. If `chunk_size` is set to 0, no feed forward chunking is done.

decoder input IDs

This input is specific to encoder-decoder models, and contains the input IDs that will be fed to the decoder. These inputs should be used for sequence to sequence tasks, such as translation or summarization, and are usually built in a way specific to each model.

Most encoder-decoder models (BART, T5) create their `decoder_input_ids` on their own from the `labels`. In such models, passing the `labels` is the preferred way to handle training.

Please check each model's docs to see how they handle these input IDs for sequence to sequence training.

? decoder models

Also referred to as autoregressive models, decoder models involve a pretraining task (called causal language modeling) where the model reads the texts in order and has to predict the next word. It's usually done by reading the whole sentence with a mask to hide future tokens at a certain timestep.

large language models (LLM)

A generic term that refers to transformer language models (GPT-3, BLOOM, OPT) that were trained on a large quantity of data. These models also tend to have a large number of learnable parameters (e.g. 175 billion for GPT-3).

M

masked language modeling (MLM)

A pretraining task where the model sees a corrupted version of the texts, usually done by masking some tokens randomly, and has to predict the original text.

multimodal

A task that combines texts with another kind of inputs (for instance images).

pipeline

A pipeline in 🤗 Transformers is an abstraction referring to a series of steps that are executed in a specific order to preprocess and transform data and return a prediction from a model. Some example stages found in a pipeline might be data preprocessing, feature extraction, and normalization.

For more details, see Pipelines for inference.

self-attention

Each element of the input finds out which other elements of the input they should attend to.

self-supervised learning

A category of machine learning techniques in which a model creates its own learning objective from unlabeled data. It differs from unsupervised learning and supervised learning in that the learning process is supervised, but not explicitly from the user.

One example of self-supervised learning is masked language modeling, where a model is passed sentences with a proportion of its tokens removed and learns to predict the missing tokens.

semi-supervised learning

A broad category of machine learning training techniques that leverages a small amount of labeled data with a larger quantity of unlabeled data to improve the accuracy of a model, unlike supervised learning and unsupervised learning.

An example of a semi-supervised learning approach is "self-training", in which a model is trained on labeled data, and then used to make predictions on the unlabeled data. The portion of the unlabeled data that the model predicts with the most confidence gets added to the labeled dataset and used to retrain the model.

sequence-to-sequence (seq2seq)

Models that generate a new sequence from an input, like translation models, or summarization models (such as Bart or T5).

supervised learning

A form of model training that directly uses labeled data to correct and instruct model performance. Data is fed into the model being trained, and its predictions are compared to the known labels. The model updates its weights based on how incorrect its predictions were, and the process is repeated to optimize model performance.

- Supervised learning is working with LABELED data (labeled data is the user telling the model what is deemed correct)

token

A part of a sentence, usually a word, but can also be a subword (non-common words are often split in subwords) or a punctuation symbol.

transfer learning

A technique that involves taking a pretrained model and adapting it to a dataset specific to your task. Instead of training a model from scratch, you can leverage knowledge obtained from an existing model as a starting point. This speeds up the learning process and reduces the amount of training data needed.

transformer

Self-attention based deep learning model architecture.

U

unsupervised learning

A form of model training in which data provided to the model is not labeled. Unsupervised learning techniques leverage statistical information of the data distribution to find patterns useful for the task at hand.

- Unsupervised learning is UNlabeled data

- RobertaTokenizerFast
  - https://huggingface.co/docs/transformers/v5.6.2/en/model_doc/roberta#transformers.RobertaTokenizerFast
  - https://huggingface.co/docs/transformers/v5.6.2/en/model_doc/roberta#transformers.RobertaTokenizerFast
  - Is_split_into_words and add_space_prefix:
    - This tokenizer will tokenize the same word differently if it's at the beginning of a sentence or not as seen in below example
    - So bc in the FiNER dataset, each string processed by tokenizer is already a single word, then we set add_space_prefix=True to make sure a word is always tokenized the same (whether at beginning of sentence or not) and as a result we have to indicate is_split_into_words=True

Construct a RoBERTa tokenizer (backed by HuggingFace's tokenizers library). Based on Byte-Pair-Encoding.

This tokenizer has been trained to treat spaces like parts of the tokens (a bit like sentencepiece) so a word will

be encoded differently whether it is at the beginning of the sentence (without space) or not:

```
>>> from transformers import RobertaTokenizer

>>> tokenizer = RobertaTokenizer.from_pretrained("FacebookAI/roberta-base")
>>> tokenizer("Hello world")["input_ids"]
[0, 31414, 232, 2]

>>> tokenizer(" Hello world")["input_ids"]
[0, 20920, 232, 2]
```

You can get around that behavior by passing `add_prefix_space=True` when instantiating this tokenizer or when you call it on some text, but since the model was not pretrained this way, it might yield a decrease in performance.

> When used with `is_split_into_words=True`, this tokenizer needs to be instantiated with `add_prefix_space=True`.

- Construct a "fast" RoBERTa tokenizer (backed by HuggingFace's *tokenizers* library), derived from the GPT-2 tokenizer, using byte-level Byte-Pair-Encoding.

  This tokenizer has been trained to treat spaces like parts of the tokens (a bit like sentencepiece) so a word will

  be encoded differently whether it is at the beginning of the sentence (without space) or not:

  ```
  >>> from transformers import RobertaTokenizerFast

  >>> tokenizer = RobertaTokenizerFast.from_pretrained("FacebookAI/roberta-base")
  >>> tokenizer("Hello world")["input_ids"]
  [0, 31414, 232, 2]

  >>> tokenizer(" Hello world")["input_ids"]
  [0, 20920, 232, 2]
  ```

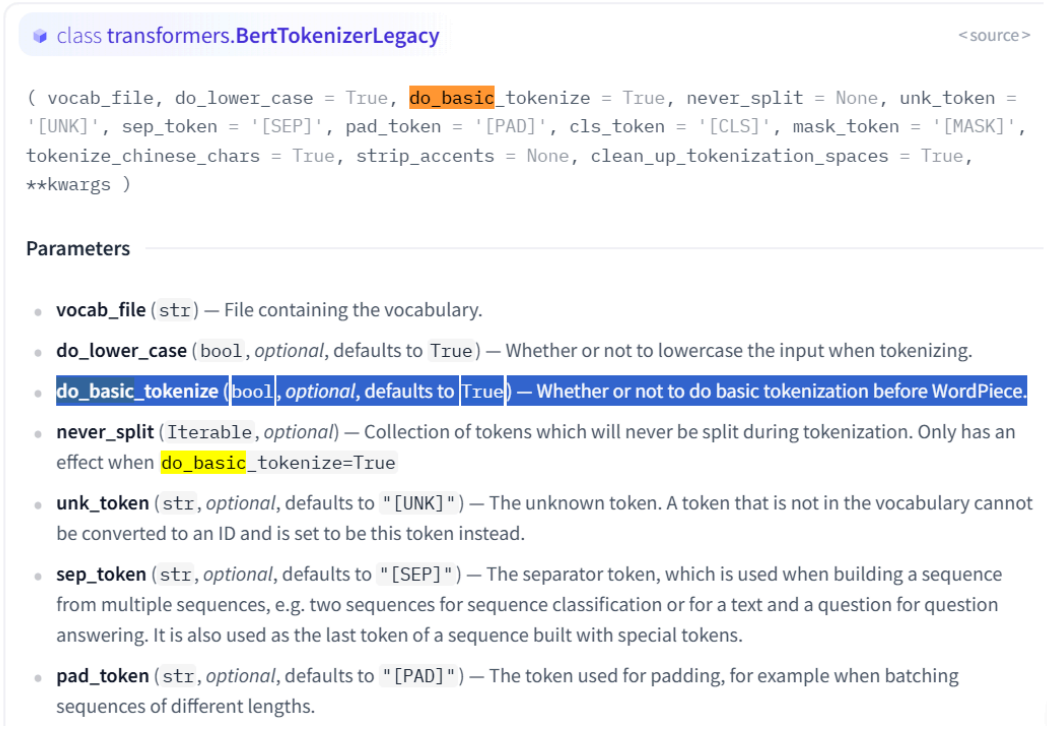
  You can get around that behavior by passing `add_prefix_space=True` when instantiating this tokenizer or when you call it on some text, but since the model was not pretrained this way, it might yield a decrease in performance.

  > When used with `is_split_into_words=True`, this tokenizer needs to be instantiated with `add_prefix_space=True`.

- Because FiNER is already tokenized to be words, we indicate is_split_into_words=True when tokenizing, and must instantiate the tokenizer with add_prefix_space=True to ensure the tokenizer doesnt break up words into

multiple tokens by adding a space at beginning of word -> FALSE!!! Words may still be tokenized into multiple subwords. We indicate add_prefix_space=True so a word no matter if it's at beginning of a sentence or not will be tokenized the same (for NER task, an entity is always the same no matter where it's located in a sentence so need to make sure tokenized the same)

BertTokenizerLegacy



- https://huggingface.co/docs/transformers/v5.6.2/en/model_doc/bert#transformers.BertTokenizerLegacy

Training

- FiNER paper found that fine tuning a pretrained (BERT) language model yielded better f1 score than zero shot prompting with LLM
- So will run pretrained hugging face model for x # epochs
- Where the “expected token” is the entity label so we use cross entropy bc take logits of confidence scores for each of the labels (CE used for multi class classification)
- NOTE we’re done with pretraining (normal language model training), so here goal is to update language model’s weights for it’s new task being NER

Data Fields

The data fields are the same among all splits.

conll2003

- `doc_idx`: Document ID (`int`)
- `sent_idx`: Sentence ID within each document (`int`)
- `gold_token`: Token (`string`)
- `gold_label`: a `list` of classification labels (`int`). Full tagset with indices:

```
{'O': 0, 'PER_B': 1, 'PER_I': 2, 'LOC_B': 3, 'LOC_I': 4, 'ORG_B': 5, 'ORG_I': 6}
```

BERT

- https://huggingface.co/docs/transformers/v5.6.2/en/model_doc/bert

Notes

- Inputs should be padded on the right because BERT uses absolute position embeddings.
  -

- https://huggingface.co/papers/1810.04805

Abstract

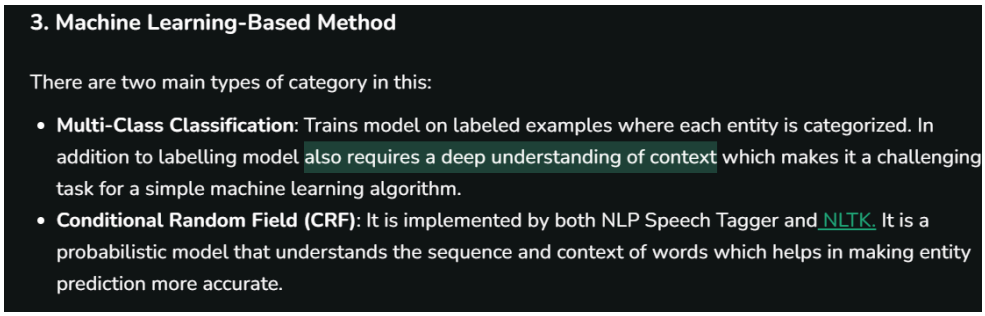
> BERT is a bidirectional transformer-based model that pre-trains on unlabeled text and fine-tunes for various NLP tasks, achieving state-of-the-art results across multiple benchmarks.
>
> AI-generated summary

We introduce a new language representation model called BERT, which stands for Bidirectional Encoder Representations from Transformers. Unlike recent language representation models, BERT is designed to pre-train deep bidirectional representations from unlabeled text by jointly conditioning on both left and right context in all layers. As a result, the pre-trained BERT model can be fine-tuned with just one additional output layer to create state-of-the-art models for a wide range of tasks, such as question answering and language inference, without substantial task-specific architecture modifications. BERT is conceptually simple and empirically powerful. It obtains new state-of-the-art results on eleven natural language processing tasks, including pushing the GLUE score to 80.5% (7.7% point absolute improvement), MultiNLI accuracy to 86.7% (4.6% absolute improvement), SQuAD v1.1 question answering Test F1 to 93.2 (1.5 point absolute improvement) and SQuAD v2.0 Test F1 to 83.1 (5.1 point absolute improvement).

RoBERTa

- https://huggingface.co/docs/transformers/v5.6.2/en/model_doc/roberta
- https://huggingface.co/papers/1907.11692

NER

- https://www.ibm.com/think/topics/named-entity-recognition
- https://arxiv.org/html/2411.05057v1 (not useful)
- https://www.geeksforgeeks.org/nlp/named-entity-recognition/
- Requires sequence understanding, so can use pretrained language model (transformer)
    - Can use a pretrained BERT like model and fine tune it on the NER task
    - Or could zero/few shot prompt with pretrained LLM (GPT)
- 

    - Unsupervised learning - unlabeled data (like pretraining language models, raw text is unlabeled)
    - Supervised learning - labeled data

    - The FiNER paper for their fine tuning method takes pretrained LM transformer to already understand sequencing and context that being attention mechanism of seeing how all other tokens affect every token which is unsupervised learning, and then supervised learning multi class classification is performed (fine tuning) by training the pretrained LM on the labeled FiNER dataset of tokens and their NER entity tags

- – **Transformers and BERT.** Transformer networks, particularly the BERT (Bidirectional Encoder Representations from Transformers) model, have had a significant impact on NER. Using a self-attention mechanism that weighs the importance of different words, BERT accounts for the full context of a word by looking at the words that come before and after it.

- **4.1 Rule-based methods**

  Rule Based NLP methods rely on hand-crafted rules, designed by domain experts and lexical patterns. In the biomedical domain Hanisch et al. (2005) uses a pre-processed synonym dictionary to identity protein mentions and potential genes in these texts. Other systems use hand-crafted semantic and syntactic rules to recognize entities. These are domain specific and are not an exhaustive list. Therefore, these systems result in high precision and low recall, and for the same reason, these cannot be used in cross domains.

-
- Decision trees Quinlan (1986), Hidden Markov Models (HMM) Rabiner and Juang (1986), Support Vector Machines and Conditional Random Fields (CRF) Lafferty et al. (2001). Maximum entropy Markov models (MEMMs) McCallum et al. (2000)

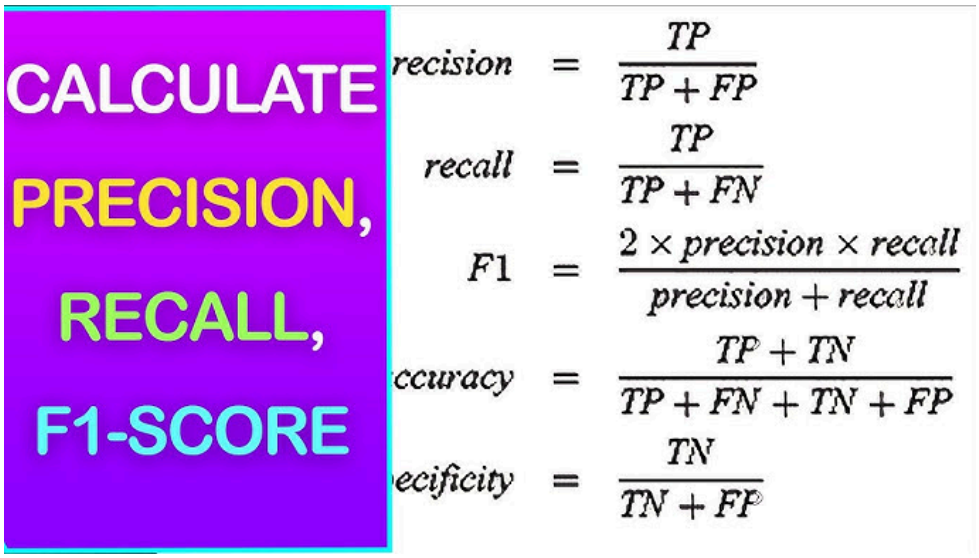
F1 Score / Metric

1. True Positive (TP) : entities that are recognized by NER and match the ground truth.
2. False Positive (FP) : entities that are recognized by NER but do not match ground truth.
3. True Negative (TN) : entities that predict the negative class correctly.
4. False Negative (FN) : entities annotated in the ground truth that are not recognized by NER.

$$Recall = \frac{TP}{TP + FN}$$

$$Precision = \frac{TP}{TP + FP}$$

$$F1 - measure = \frac{2 * Precision * Recall}{Recall + Precision}$$



Actual Classes

Predicted Classes	a	b	c	d
a	50	3	0	0
b	26	8	0	1
c	20	2	4	0
d	12	0	0	1

- Confusion matrix for multi class classification

- https://medium.com/@sreenilarajesh/confusion-matrix-for-multiclass-classification-fe7b6901a541

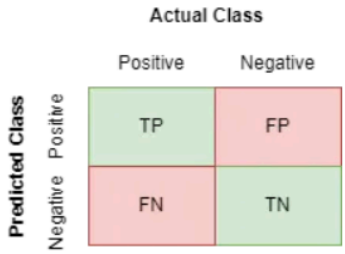


Fig 1: Confusion Matrix for Binary Classification

> Unlike binary classification, the classes in multi-class classification cannot be directly represented as positive or negative of a value. Hence, we cannot compute TP (True Positive), TN (True Negative), FP (False Positive), and FN (False Negative) values in a straightforward manner. Instead, we need to calculate TP, TN, FP, and FN for ***each class.***

  -
  - Read above article to understand confusion matrix and F1 for multi class classification
  - explains how for non-binary classification (more than 2 labels), each label calculates its own f1 score and doing so will get its own unique TP, FP, FN, and TN in the confusion matrix for each class
  - confusion matrix values are constant, but each class will have different spots in the confusion matrix be TP, FP, FN, and TN. It is in the perspective of the class you're trying to calc the F1 for
    - TP would be all the predictions that were correctly predicted as the label you're calculating F1 for
    - FP would be all of the predictions that were incorrectly predicted as the label you're calculating F1 for
    - TN would be all of the predictions that were not predicted as the label you're interested in and whose true label is not that label either
    - FN would be all of the predictions that were not predicted as the label you're interested in but their true label is the label you're interested in

Coding

- Goal is to display in depth understanding
    - Good documentation
    - Visual umdertstnaidng (outputs)
    - Metrics / logging
- Add LoRA?
- To do methods
    - Load_data
        -
```
import pandas as pd

a = pd.DataFrame([[0, 24]], columns=["bob", "some"])
print(a, "\n")
d = {0: "a", 1: "b", 2: "c", 3: "d"}

a.bob = a.bob.map(d)
print(a)
```
```
   bob  some
0    0    24

  bob  some
0   a    24
```
        - Gold_label column mapped from int to its string representation

[2] ✓ 0s

```python
import pandas as pd

a = pd.DataFrame([[0, 24], [3, 4], [3, 111], [1, 24], [2, 24]], columns=["bob", "some"])
print(a, "\n")
d = {0: "ss", 1: "b", 2: "r", 3: "d"}

a.bob = a.bob.map(d)
print(a)

label_list = list(a.bob.unique())
label_list.sort()
str_to_int = {l: i for i, l in enumerate(label_list)}
int_to_str = {i: l for (l, i) in str_to_int.items()}

print(label_list)
print(str_to_int)
print(int_to_str)
```

```
   bob  some
0    0    24
1    3     4
2    3   111
3    1    24
4    2    24

  bob  some
0  ss    24
1   d     4
2   d   111
3   b    24
4   r    24
['b', 'd', 'r', 'ss']
{'b': 0, 'd': 1, 'r': 2, 'ss': 3}
{0: 'b', 1: 'd', 2: 'r', 3: 'ss'}
```

-
  - Also label's dictionary are reindexed by sorting them by their string (alphabetical)
  - Assuming each train, val, and test.csv have one of each type of label this should be fine which they do verify with the below asserts:

```python
503     def grid_search_bert(self):
504         train_dataset, train_str2int, train_int2str = self.load_data(self.train_path, self.int2str)
505         val_dataset, val_str2int, val_int2str = self.load_data(self.val_path, self.int2str)
506         test_dataset, test_str2int, test_int2str = self.load_data(self.test_path, self.int2str)
507         assert train_str2int == val_str2int == test_str2int, f"Labels are mismatching"
508         assert train_int2str == val_int2str == test_int2str, f"Labels are mismatching"
509 
510         num_labels = len(train_int2str)
```

  - So can redo the label's dictionary as long as it's consistent for all of the .csv's
- Tokenizer_output.word_ids

hugginf face tokenizer what is .word_ids

AI Overview

In a Hugging Face Fast Tokenizer, `.word_ids()` is a method that maps every generated **token** back to the index of the **original word** it came from. GitHub +1

This is essential because modern tokenizers often split a single word into multiple subwords (e.g., "transformer" becoming ["transform", "##er"]). The `.word_ids()` list allows you to track which subwords belong to the same original word. GitHub +2

Key Features of `.word_ids()`

- **Mapping**: Each element in the returned list corresponds to a token at that same position in your input IDs.
- **Word Indices**: If multiple tokens are part of the same word, they will all share the same word index.
- **Special Tokens**: Tokens like `[CLS]`, `[SEP]`, or padding are mapped to `None`.
- **Requirement**: This method is only available for "Fast" tokenizers (those backed by Rust). Hugging Face +4

Example

If you tokenize the sentence "Hugging Face is great":

Token	Hugging	Face	is	great
Token Index	0	1	2	3
Word ID	0	1	2	3

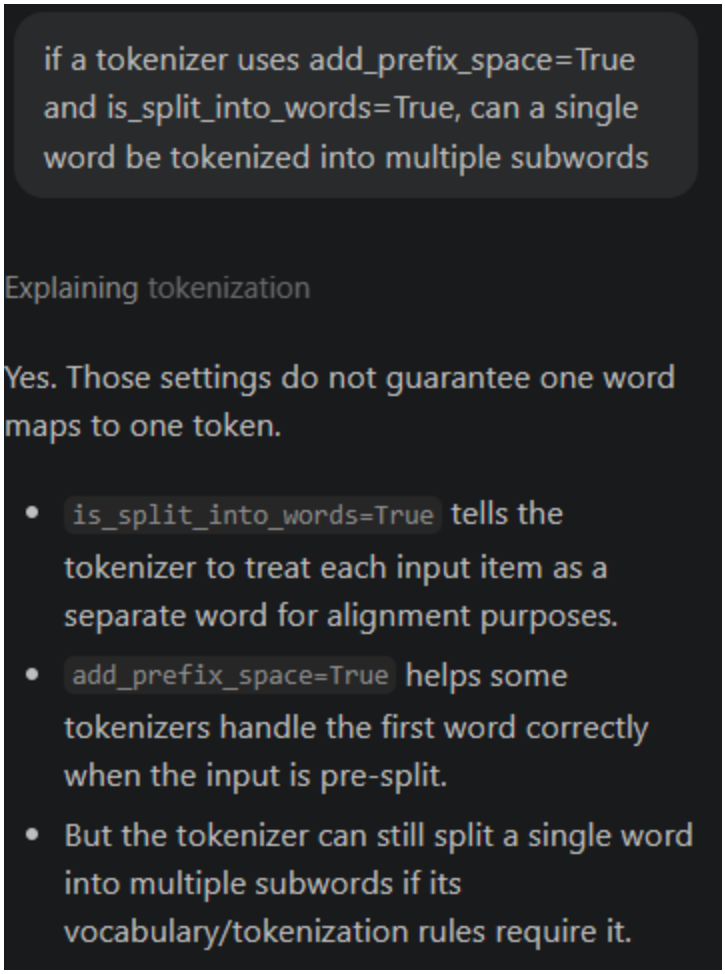
If "Hugging" was split into ["Hug", "##ging"]:

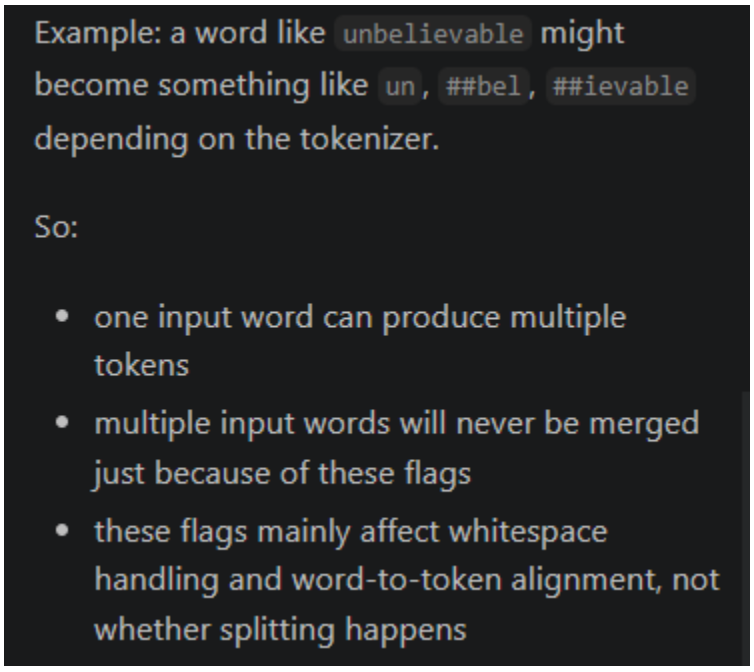
Token	[CLS]	Hug	##ging	Face	...
Word ID	None	0	0	1	...

Common Use Case

The primary use for `.word_ids()` is **Token Classification** tasks, such as Named Entity Recognition (NER). It ensures that if a word is labeled as a "Person," all its individual subword tokens are correctly associated with that label during training. Hugging Face +2

- VERY IMPORTANT: is_split_into_words=True and add_predix_space=True
  - Words may still be tokenized into multiple subwords as seen in above example

- 

- 

    - add_prefix_space=True

hugging face tokenizer do you use add_predix_space when your sequence is a sentence?

AI Overview

You should typically set `add_prefix_space=True` (or handle it by adding a leading space manually) if your tokenizer uses Byte-Pair Encoding (BPE) and you are tokenizing a single sentence that doesn't start with a space. This ensures the first word is treated the same as subsequent words that include a leading space. Hugging Face Forums +1

Key Considerations for `add_prefix_space`:

- **Purpose:** It adds a space at the beginning of the string, which is necessary for BPE tokenizers (like BERT, GPT, RoBERTa) that treat words (or subwords) as beginning with a whitespace character.
- **When to use:** When your input is a single string or list of sentences and you're not pre-tokenizing, `add_prefix_space=True` provides more consistent, expected tokenization, especially for the very first token.
- **Alternatives:** Instead of relying on the argument, you can manually add a leading space to your input string (`" " + sentence`) and set `add_prefix_space=False` .
- **When to avoid:** If you are using a tokenizer that is *not* based on whitespace/BPE (e.g., some SentencePiece implementations), this might not be required. Hugging Face Forums +4

Using it ensures the tokenizer doesn't treat the first word differently than the rest of the sentence, avoiding issues where the first word is split differently. GitHub

- Main
- Fine tune
- test